

AI-Powered Mock Interview Platform with Mode-Driven Interviewer Behavior and Evidence-Based Feedback

Project Team: sudo

Team 27

Başak Güney
211101051
bguney@etu.edu.tr

Ecesu Bozkurt
201301005
ecesu.bozkurt@etu.edu.tr

İnci Sıla Kaleli
201301014
incisilakaleli@etu.edu.tr

Abstract—This project implements an AI-powered mock interview platform designed to provide a realistic, structured, and repeatable interview-practice experience. The implementation is organized as a React and TypeScript client, an Express-based Node.js orchestration layer, and a Python analysis service responsible for audio, transcript, and vision-related processing. The delivered platform includes configuration and consent handling, preview-question generation, WebRTC-based live interviewing, supportive-mode hints and instant feedback, CV upload and candidate-brief extraction, incremental answer-audio ingestion, multimodal report generation, persistent report archival, history analytics, token-cost estimation, and a Docker-based multi-service deployment setup. Unlike a static chatbot prototype, the current project implements a complete interview lifecycle with explicit session state, backend guardrails, asynchronous artifact completion, and evidence-based feedback presentation. This report is based only on the source code and automated tests currently present in the project.

Index Terms—AI interview simulator, WebRTC, React, Express, FastAPI, multimodal analysis, feedback systems

I. INTRODUCTION

Interviews play a critical role in early career development, as they are widely used to evaluate candidates' technical knowledge, communication skills, and overall suitability for a position. For students and recent graduates, effective interview preparation is therefore an important step toward improving employment opportunities and professional confidence. However, many candidates lack access to realistic, repeatable, and feedback-oriented practice environments [18], [19].

Traditional preparation methods, such as reviewing frequently asked questions or conducting informal mock interviews, can help candidates become familiar with interview formats, but they often provide only limited benefit. In most cases, these approaches do not offer structured, objective, or measurable feedback. As a result, candidates may struggle to assess the quality of their answers, the clarity of their communication, and their ability to respond effectively under pressure.

Recent advances in artificial intelligence have created new opportunities for more interactive and adaptive interview preparation systems. AI-based tools can simulate interviewer behavior, generate context-aware questions, and evaluate candidate responses at scale [16], [20]. However, many existing solutions primarily focus on question generation or general answer scoring, while offering limited insight into broader communication behaviors and overall interview performance [7], [9].

This project presents an AI-powered mock interview platform designed to provide a realistic, structured, and repeatable interview practice experience. The platform enables users to participate in voice-based interview sessions with an AI interviewer and receive evidence-based feedback on both response quality and communication patterns. By combining conversational AI with structured session management and performance evaluation, the system aims to make interview preparation more accessible, personalized, and effective for early-career candidates.

A. Purpose

The purpose of this project is to develop an interview practice platform that supports realistic simulation and meaningful performance evaluation. The system is intended to help users improve not only the content of their answers but also the way they communicate during an interview.

More specifically, the platform aims to provide a configurable interview environment based on role, difficulty, and company or industry context, while also generating evidence-based post-session feedback from multiple signals. In this way, the project goes beyond simple question-answer interaction and focuses on supporting measurable user improvement through structured practice.

B. Scope

The scope of the implemented platform includes the complete lifecycle of a mock interview session, from preparation to post-session reporting. Based on the current

system design and project structure, the platform covers the following functionalities:

- session setup with required microphone and camera consent,
- optional CV upload and candidate-specific briefing generation,
- preview-question generation before the interview session,
- real-time voice-based interview execution,
- supportive and neutral interviewer modes,
- incremental upload and processing of candidate answer audio,
- transcript, audio, and visual analysis of interview performance,
- report generation and archival of session artifacts,
- history and trend views across previous sessions.

The current scope is limited to interview-session orchestration, multimodal analysis, and reporting functionalities. A persistent relational database and a full user-account management subsystem are not included in the same codebase. Instead, active session data is managed in memory, while completed report artifacts are archived in the server-side reports directory [2], [4].

II. PROBLEM DEFINITION

Conventional interview preparation methods are often insufficient for candidates who need realistic speaking practice and objective feedback. Static question lists may help users review common interview topics, but they do not replicate live interaction, follow-up questioning, or time-constrained spoken responses. Similarly, informal mock interviews can provide useful impressions, yet they are difficult to standardize, repeat consistently, and evaluate objectively [18], [19].

Another important limitation is the lack of real-time support in many interview-preparation tools. In many cases, feedback is provided only after the interview has ended, which limits the candidate’s ability to recognize weaknesses during the interaction itself. As a result, users may complete practice sessions without receiving timely guidance on how to adjust their response structure, speaking flow, or communication behavior while the interview is still in progress [7], [9].

This limitation becomes more visible in the context of Turkish interview practice. Tools and examples that work well for English-centered interview preparation are not always sufficient for Turkish-speaking candidates, especially when speech clarity, fluency, and role-specific communication need to be evaluated in Turkish. This creates a gap between generic AI assistance and the needs of users who require a Turkish-speaking interviewer, Turkish transcript handling, and feedback that reflects spoken performance rather than only text quality.

From a system-design perspective, the problem is not limited to generating interview questions. An effective

interview-preparation platform must also manage interview flow, preserve conversational context across turns, adapt to different interview styles, and produce actionable evaluation based on multiple forms of evidence. These forms of evidence may include spoken responses, transcript content, delivery patterns, and visible engagement signals [15], [17].

The problem addressed in this project is therefore the design and implementation of a structured AI-based interview system that can simulate a realistic interview process while also producing comprehensive and evidence-based feedback. To address this need, the implemented platform combines conversational AI with rule-based orchestration and multimodal post-session analysis. Rather than treating the interview as a single prompt-response exchange, the system models a complete session lifecycle that includes preparation, live interaction, evaluation, and review [1]–[4].

III. BACKGROUND RESEARCH, SIMILAR DESIGNS, AND ENGINEERING PRINCIPLES

A. Background Research and Similar Designs

Background research indicates that AI-supported interviewing systems are increasingly used to automate parts of candidate screening, structured interviewing, and interview preparation. Commercial examples such as HireVue and Pymetrics demonstrate that AI can be integrated into interview-related workflows for question delivery, response analysis, and candidate assessment at scale [18], [19]. Academic and professional discussions also show that AI-enabled interviews raise important questions about realism, fairness, transparency, and user trust [7]–[10].

Compared with such systems, the implemented project focuses on interview practice rather than employment decision-making. This distinction is important from both a technical and ethical perspective. The system is designed to help users improve through repeated practice, supportive guidance, and evidence-based post-session reporting, rather than to make automated hiring decisions. Nevertheless, similar-design research remains relevant because it highlights both the opportunities and risks of AI-mediated interview environments [7], [11].

B. Component Information

The project combines several component classes commonly found in modern AI-enabled systems. The conversational layer relies on large language models for question generation, interviewer behavior, and narrative feedback support [15], [20]. The signal-analysis layer uses established machine-learning and deep-learning techniques for transcript, audio, and vision processing [16], [17]. In addition, the web platform integrates browser media capture, WebRTC communication, and modular backend orchestration to support low-latency interactive use.

These component choices reflect practical engineering trade-offs. LLMs provide flexibility and naturalness in

interaction, whereas deterministic backend guardrails improve control, consistency, and safety. Similarly, multi-modal analysis broadens the evidence base for feedback, but also increases system complexity and introduces additional privacy and fairness considerations [8], [10].

C. Basic Engineering Principles

Several basic engineering principles guide the implemented design. First, modularity is preserved by separating the client, orchestration backend, and analysis service into distinct layers. Second, robustness is improved through validation, fallback logic, duplicate suppression, and graceful degradation under partial-service failure. Third, maintainability is supported through explicit session lifecycle management and well-separated responsibilities across components. Finally, user-centered engineering considerations are reflected in supportive-mode guidance, consent enforcement, and evidence-oriented feedback presentation.

These principles are consistent with broader software-engineering and AI-system design practice, where systems are expected not only to function correctly, but also to remain understandable, extensible, and safe under realistic operating conditions [16], [17].

IV. SYSTEM OVERVIEW AND SOLUTION APPROACH

A. System Perspective

The system is implemented as a layered architecture with three major runtime zones. The first zone is a web client written in React 19, TypeScript, and Vite. The second zone is an Express server that manages validation, orchestration, persistence, routing, and OpenAI-facing communication. The third zone is a Python analysis service that performs audio, transcript, and vision-related computation. This makes the platform a full-stack application with both synchronous and asynchronous feedback paths. In addition to this logical separation, the repository includes a Docker Compose-based deployment structure that runs the client, server, and Python analysis service as distinct containers with health-checked startup ordering and shared report storage [6].

B. Solution Approach

The implemented solution adopts a hybrid approach. Large language models are used to generate questions, maintain the conversational flow, and interpret several forms of feedback, while deterministic backend rules keep the interview process bounded and consistent [15], [20]. This combination is important because a purely prompt-driven flow can become unstable in longer sessions, whereas a fully rule-based system would not provide the flexibility needed for natural conversation, role-aware questioning, and mode-dependent interviewer behavior.

The system also separates interaction management from analysis. During the session, the platform focuses on real-time interviewing, supportive guidance when enabled, and

artifact collection. After or near the end of the session, transcript, audio, and vision signals are analyzed through dedicated processing layers and then aggregated into a unified report. This separation improves modularity and makes it possible to extend one part of the platform without redesigning the entire workflow.

Another important aspect of the implementation is the use of fallback and degradation mechanisms across multiple layers. The system includes rule-based safeguards for invalid session states, consent enforcement before interview start, duplicate-aware handling of incremental audio uploads, transcript fallback when direct transcript evidence is incomplete, provisional audio reporting while final analysis is still pending, and graceful degradation when external analysis services become unavailable or time out. In the vision pipeline, MediaPipe-based detection is supported by an OpenCV fallback path so that the reporting flow can continue even when the preferred detector is not available. These mechanisms improve robustness by ensuring that the interview process and post-session reporting remain functional under partial-data and partial-service conditions.

C. Product Functions

The codebase currently implements the following product-level functions:

- Collect interview configuration fields including first name, last name, gender, role, company or industry, domain interest, difficulty, and interaction mode.
- Enforce microphone and camera consent before the session can proceed.
- Generate preview questions tailored to HR or technical interviews.
- Conduct a live interview with an AI interviewer and display real-time audio and transcript updates during the session.
- Provide supportive-mode hints and toast feedback during the interview.
- Upload candidate answer audio incrementally while the interview is running.
- Analyze transcript quality, answer relevance, audio delivery, and visual presentation.
- Build a unified report containing scores, recommendations, transcript text, and analysis status flags.
- Re-open previous reports and show cross-session trends in the history screen.

D. User Workflow

The live workflow begins at the setup screen. The setup stage constructs the initial interview configuration and validates it through a consent-aware form flow. The start action remains disabled until all mandatory fields are populated and both microphone and camera consent are granted. If the user uploads a PDF CV, the client reads the file, encodes it as Base64, and attaches it to the outgoing session request [1]. This allows the system to move beyond

generic interviewing and prepare a more context-aware session.

The preview screen performs client-side device checks. Video testing is based on a local camera stream rendered inside a preview panel. Audio testing uses the browser audio-processing stack and an analyser configuration. The screen also requests three preview questions from the backend and exposes editable candidate-summary fields if a backend-generated summary is available.

The interview screen then opens the actual runtime path. It starts the live camera, starts browser-side vision sampling, creates the WebRTC connection for the interviewer session, maintains a rolling transcript, captures interviewer audio clips, and periodically flushes candidate answer audio every 2.5 seconds. During this stage, the platform behaves as a voice-based interview system rather than a text chatbot: the user interacts through speech, the interviewer responds in realtime, and supportive-mode interventions can be injected when that operating mode is active. When the user finishes the interview, the client sends the transcript and all candidate-answer audio artifacts to the backend and transitions to the feedback page with an optimistic placeholder report until asynchronous analysis becomes ready.

The feedback screen polls the backend report endpoint and updates the visible analysis status when transcript, audio, or vision artifacts become ready. The history screen lists archived sessions, retrieves metadata from stored reports, filters by mode and interview type, and visualizes cross-session score patterns with chart components. Table I summarizes the main screens and their responsibilities in the implemented client workflow.

V. SYSTEM ARCHITECTURE AND TECHNICAL DESIGN

A. Architectural Overview

The system follows a three-tier design: (i) a browser client responsible for configuration capture, device diagnostics, realtime interview interaction, and visualization; (ii) a Node.js/Express orchestration backend that manages the session lifecycle, policy checks, realtime signaling, and artifact persistence; and (iii) a Python analysis service that produces transcript, audio, and vision analysis artifacts. Persisted artifacts are stored on disk under `server/reports` to support report retrieval and cross-session history without a database.

B. Technology Stack

Client. The client is built with React (19.2.0), TypeScript (~5.9.3), and Vite (~7.2.4). UI composition uses Tailwind CSS (with PostCSS and `tailwindcss-animate`) and Radix UI primitives. Class composition and styling utilities include `clsx`, `class-variance-authority`, and `tailwind-merge`. For UX and visualization, the client uses Lucide React, Framer Motion, and Recharts.

Backend. The backend is a Node.js server using Express

(4.19.2) with `cors` and `dotenv`. It uses `pdf-parse` for CV text extraction and `ffmpeg-static` to support audio conversion workflows. Automated checks follow Vitest and Supertest-based patterns.

Python analysis service. The Python service uses FastAPI and Uvicorn with Pydantic and `python-multipart`. Core analysis dependencies include PyTorch, TorchAudio, Transformers, Librosa, NumPy, SoundFile, OpenCV (headless), and MediaPipe. The stack also includes `qdrant-client[fastembed]` for vector search/embeddings, and `requests/matplotlib` for HTTP utilities and visualization/debug support [17].

OpenAI services. OpenAI is integrated as a runtime dependency: realtime interviewing is configured with `gpt-realtime-mini`; live input transcription uses `gpt-4o-mini-transcribe`. Preview question generation and several post-hoc narrative/report steps use `gpt-4o-mini`. A Node-side transcript evaluation component uses `gpt-4.1-mini`, and candidate-audio transcription fallback uses `gpt-4o-transcribe` [20].

Containerization and deployment. The repository also includes Dockerfiles for the client, backend, and Python analysis service, together with a `docker-compose.yml` file that defines the multi-container runtime, shared report volume, service health checks, and environment-variable-based configuration [6]. Table II groups the visible technologies by architectural layer.

C. Client Layer

The client implements navigation using a route-key model rather than a dedicated routing library. A navigation snapshot is stored in browser history state, which preserves the active route, the current session configuration, and the latest report payload during back/forward navigation. The Setup screen collects identity and interview configuration and enforces mandatory microphone and camera consent before proceeding. An optional CV PDF can be uploaded; the client reads the file and forwards the encoded payload as part of session configuration. The Preview screen performs device diagnostics. For audio, it uses an `AnalyserNode` with `fftSize=256` and derives a normalized live level from average frequency-bin energy. For video, it requests a 1280×720 stream and renders a live preview. The screen triggers backend-driven preview question generation and allows editing of candidate-brief text fields when CV-derived summaries are present. The Interview screen connects to the realtime session, maintains transcript state, and periodically flushes incremental candidate answer audio uploads every 2.5s while active, with a final flush during session termination. It also integrates supportive-mode UI elements (hints and instant feedback toasts) when enabled. After the interview ends, it closes realtime/media resources and transitions to the feedback view with an optimistic placeholder report while asynchronous analysis completes. The Feedback screen polls for report readiness and progressively renders tran-

Table I
IMPLEMENTED SCREENS AND RESPONSIBILITIES

Screen	Main responsibility in the codebase
Setup	Collect configuration, consent, and optional CV upload
Preview	Test camera and microphone, request preview questions, edit candidate brief
Interview	Run live session, collect transcript and answer audio, sample vision frames
Feedback	Poll report readiness and display transcript, audio, and vision analyses
History	List archived sessions, open prior reports, show score and trend summaries

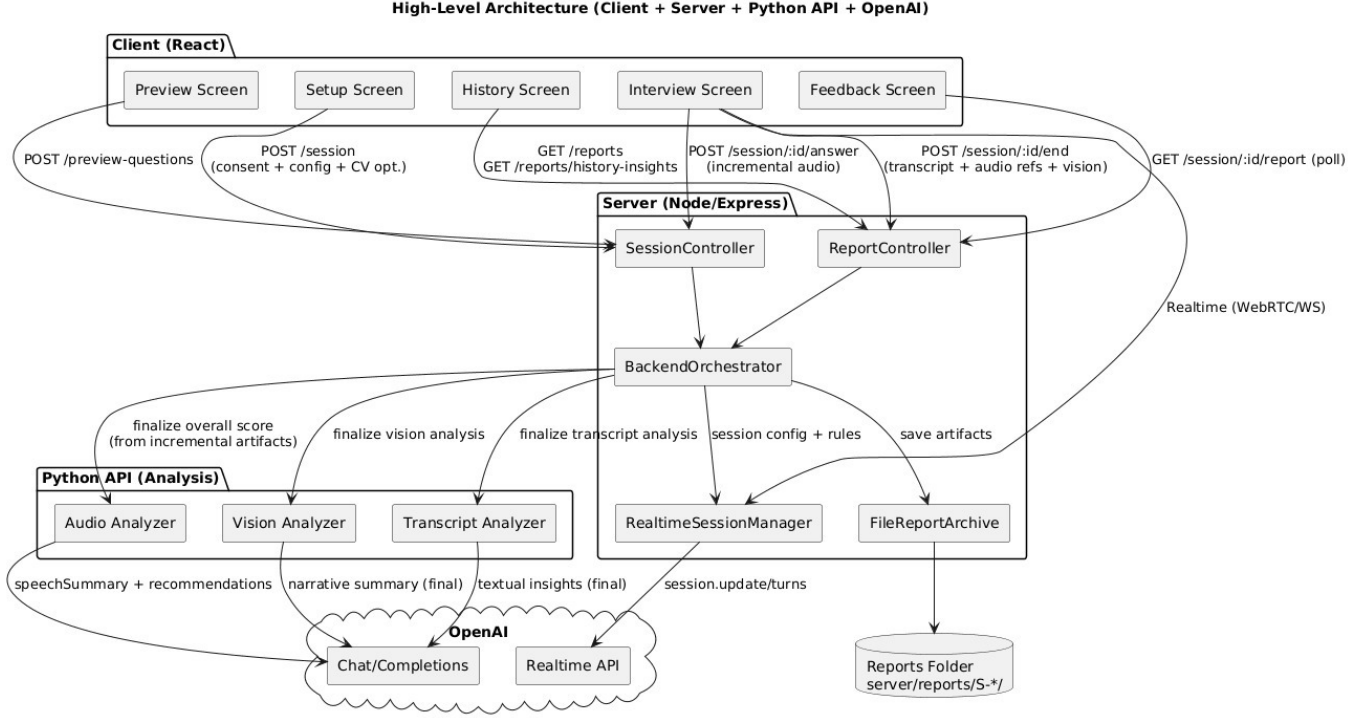


Figure 1. High-Level Architecture

script, audio, and vision artifacts. The History screen lists archived sessions and enables opening prior reports, with trend and distribution visualizations.

D. Backend Layer

The backend is an Express server initialized with CORS, JSON parsing, and a plain-text body parser that explicitly supports `application/sdp` for realtime signaling payloads. It also provides a backward-compatible `/log` endpoint for transient client RTC/debug logs and mounts the main session router. The router exposes endpoints for session creation, session start, configuration updates, consent persistence, preview question generation, supportive-mode utilities (hints, instant feedback, and usage recording), realtime SDP offer/answer exchange, incremental answer ingestion, vision-frame ingestion, session finalization, report retrieval, archived report listing, and history insight computation. Core coordination is implemented

in `BackendOrchestrator`. Active sessions and reports are stored in in-memory repositories, while persistent artifacts are written through a filesystem-backed archive. Consent and start-state validation are enforced through `GuardrailsEngine` (microphone and camera consent are mandatory; invalid start states are rejected). During an active session, the backend maintains runtime structures for incremental candidate audios, saved audio file references, analyzed-audio bookkeeping, and a vision sub-state that aggregates detector-derived signals and counters. Table III lists the main backend endpoints and their current responsibilities.

E. Python Analysis Layer

The Python analysis layer is implemented as a standalone FastAPI service (served via Uvicorn) that provides HTTP endpoints for transcript, audio, and vision processing. Requests are modeled and validated with Pydantic,

Table II
TECHNOLOGY STACK BY LAYER

Layer	Technologies visible in the project
Client	React 19, React DOM, TypeScript, Vite, Tailwind CSS, PostCSS, Radix UI, Lucide React, Framer Motion, Recharts
Client tooling	ESLint, TypeScript ESLint, Testing Library, jsdom, Vitest
Backend	Node.js, Express, cors, dotenv, pdf-parse, ffmpeg-static
Containerization / deployment	Docker, Docker Compose, service-specific Dockerfiles, shared report volume, service health checks
LLM services	OpenAI Realtime API (<code>gpt-realtime-mini</code>), OpenAI Chat Completions (<code>gpt-4o-mini</code> , <code>gpt-4.1-mini</code>), OpenAI Transcription (<code>gpt-4o-transcribe</code> , <code>gpt-4o-mini-transcribe</code>)
Backend testing	Vitest, Supertest
Python API	FastAPI, Uvicorn, Pydantic, python-multipart, requests, matplotlib
ML and signal processing	PyTorch, TorchAudio, Transformers, Librosa, NumPy, SoundFile, OpenCV (headless), MediaPipe
Vector search / embeddings	qdrant-client[fastembed]

and `python-multipart` is used for file-compatible payload handling where required. The service relies on ML and signal-processing libraries including PyTorch, TorchAudio, Transformers, Librosa, NumPy, SoundFile, OpenCV (headless), and MediaPipe to compute structured metrics and scores from session artifacts. The project also includes `qdrant-client[fastembed]` to support embedding-based retrieval components used by analysis utilities. At session finalization, the Node.js backend invokes this service to produce finalized analysis outputs (e.g., transcript/audio/vision report artifacts) that are persisted into the session’s archive directory and later consumed by the UI and history computations.

F. External Interfaces

The system exposes four primary interface surfaces. First, the browser interface uses MediaDevices and Web Audio APIs for capture and diagnostics, plus History State for lightweight routing and state persistence. Second, a REST interface connects the client to the Express backend for session lifecycle operations, supportive-mode utilities, incremental artifact ingestion, and report retrieval. Third, a WebRTC-based interface connects the client, backend SDP exchange, and the OpenAI realtime service for low-latency interviewing, including a data channel for model events. Fourth, an HTTP bridge connects the Node.js

backend to the Python FastAPI analysis service to trigger transcript/audio/vision analysis and to materialize finalized report artifacts that are later served to the client. This separation ensures that capture, orchestration, external model calls, and deterministic analysis remain decoupled while still producing a unified report view.

G. Realtime Communication Path

Realtime interviewing is implemented with WebRTC. The client creates an `RTCPeerConnection` configured with a public STUN server, adds a receive-only audio transceiver for interviewer audio, opens a data channel (`oai-events`) for model events, and injects the local microphone track into the peer connection. The backend exchanges SDP via `/realtime/offer` and configures the realtime session, including server-side voice activity detection (VAD) with threshold-based turn detection and automatic response creation. The project contains two VAD configurations: a default server policy (`threshold=0.95`, `prefix_padding_ms=300`, `silence_duration_ms=1500`) and a client-side `session.update` that sets `silence_duration_ms=700`. Transcript events are normalized into stable transcript entries with role, timestamp, and source/model metadata, and lightweight fingerprinting is applied to reduce duplication.

Table III
MAIN BACKEND SERVICE CAPABILITIES (CURRENT IMPLEMENTATION)

Backend capability (endpoint)	Purpose
Session creation (POST /session)	Create a session, persist initial configuration, optionally accept an SDP offer and return an SDP answer.
Session start (POST /session/:sessionId/start)	Validate start state and generate the first interviewer question.
Session configuration update (PATCH /session/:sessionId/config)	Update session configuration prior to the interview and persist updated config.
Consent update (PATCH /session/:sessionId/consent)	Persist consent state; microphone and camera consent are enforced as mandatory.
Preview question generation (POST /preview-questions)	Generate exactly three preview questions.
Supportive live hints (POST /session/:sessionId/supportive/hints)	Generate short live hints to guide the candidate's next answer.
Supportive instant feedback (POST /session/:sessionId/supportive/feedback)	Generate a single post-answer coaching toast (question + answer).
Usage recording (POST /session/:sessionId/usage)	Persist token usage emitted by supportive features and other tracked components.
Realtime offer handling (POST /realtime/offer)	Exchange SDP and return the realtime SDP answer payload.
Incremental answer ingestion (POST /session/:sessionId/answer)	Upload one candidate answer audio incrementally.
Vision frame ingestion (POST /session/:sessionId/vision/frame)	Upload one sampled video frame for analysis accumulation.
Session finalization (POST /session/:sessionId/end)	End the session, persist artifacts, and trigger report generation and Python analysis finalization.
Session finalization (legacy) (POST /session/:sessionId/report)	Backward-compatible alias of session finalization.
Report retrieval (GET /session/:sessionId/report)	Retrieve the assembled report view and associated analysis artifacts.
Archived report listing (GET /reports)	List stored report summaries from the filesystem-backed archive.
History insight generation (GET /reports/history-insights)	Compute cross-session summaries and trends using archived artifacts.
Client log endpoint (POST /log)	Backward-compatible endpoint for transient client RTC/debug logs.

H. Persistence and Report Artifacts

Session artifacts are persisted under `server/reports/S-*/`. The archive includes: `session_config.json` (session metadata), `transcript.txt` (plain transcript text), `transcript_report.json` (structured transcript analysis), `audio_segments.json` (incremental audio model outputs/segments), `audio_report.json` (final audio interpretation), `vision_frames.json` (sampled vision frames/metrics), and `vision_report.json` (final vision report). These artifacts are loaded by the backend to serve report views and to compute cross-session history insights.

VI. NEW TOOLS AND TECHNOLOGIES USED DURING THE COURSE

The project required the team to work with several tools and technologies that extend beyond conventional web application development. At the client layer, modern frontend development practices were applied through React, TypeScript, Vite, Tailwind CSS, and component-driven UI composition. At the communication layer, WebRTC was used to support low-latency live interviewing, which

introduced practical concepts such as peer connections, SDP exchange, audio tracks, and event channels.

On the backend side, the project used Node.js and Express not only for standard API routing but also for realtime coordination, artifact persistence, and orchestration logic. In addition, the Python analysis service introduced FastAPI-based service design together with machine-learning libraries such as PyTorch, TorchAudio, Transformers, Librosa, OpenCV, and MediaPipe. This gave the project a multimodal processing capability that combines conventional software engineering with AI-enabled analysis workflows [16], [17].

Another important technology adopted during the course was Docker-based containerization. The project was dockerized so that the client, backend, and Python analysis service could be built and executed as separate services under Docker Compose. This provided practical experience with container images, service orchestration, shared volumes, environment-variable management, and health-checked startup dependencies in a multi-service application [6]. The Docker setup was especially useful for keeping the Python analysis stack reproducible despite its heavier dependencies.

The course project also required practical use of OpenAI services, including realtime interaction, speech transcription, question generation, and report-oriented prompting. This exposed the team to modern LLM-based system integration, structured prompting, token-aware service usage, and the practical engineering challenges of combining generative AI components with deterministic control logic [20]. As a result, the project served not only as an implementation exercise, but also as an opportunity to work with current full-stack, realtime, and AI-assisted software technologies.

VII. MODE-DRIVEN INTERVIEWER BEHAVIOR AND PERSONALIZATION

Question generation and supportive features are implemented through a dedicated AI service layer (`AIServiceGateway`) and structured prompt templates (`PromptTemplates`). The implementation explicitly distinguishes between HR and technical interviews. For preview generation, both tracks request exactly three questions. HR preview generation incorporates a curated HR question bank loaded from `hr_question_bank.json` and instructs the model to maintain theme diversity across behavioral categories such as motivation, teamwork, conflict management, feedback, and time management. Technical preview generation conditions on role, company/industry, domain interest, and difficulty, and requests three domain-constrained technical questions with explicit diversity constraints (e.g., avoid paraphrases and avoid over-focusing on a single project), optionally grounding questions with CV-derived evidence when available.

The live interviewer is governed by explicit session instructions assembled by `PromptTemplates.sessionInstructions(cfg)` and injected into the realtime session via the session builder. These instructions enforce Turkish-only output and prescribe an interview macro-structure (OPENING → QUESTION LOOP → CLOSING). They also include anti-repetition and diversity constraints, such as limiting how long the interviewer may stay on the same project/example and requiring thematic switching across the interview. Mode selection affects interpersonal style: in Supportive mode the interviewer may provide brief guidance when the candidate is stuck, whereas in Neutral mode the interviewer remains more formal and less directive. In addition, content-domain rules are specialized by interview type (HR vs. Technical), so interviewer behavior is driven along two orthogonal dimensions: (i) interpersonal style (Supportive vs. Neutral) and (ii) content domain (HR vs. Technical).

CV-driven personalization is implemented as a first-class feature in the filesystem-backed archive layer (`FileReportArchive.saveCvFile`). When a PDF CV is uploaded, the backend persists the original PDF under the session archive, extracts text using a PDF parsing utility, and produces a structured CV representa-

tion (`cv.json`). The backend may additionally produce an optional Turkish translation file (`cv_tr.txt`) using a dedicated translation helper (`cv_translator.py`, default model `Helsinki-NLP/opus-mt-en-tr`). From the structured CV, the system derives a compact candidate brief (`candidate_brief.json`) containing both technical and HR-oriented summaries and highlight lists (e.g., education, experience, projects, skills, HR experience highlights, and inferred behavioral focus areas). This brief is reused in preview prompting and is also incorporated into live session instructions, with explicit guidance to treat CV statements as hypotheses to be validated through the candidate’s spoken answers.

Supportive mode adds two runtime services invoked by the client. First, live hint generation: when the interviewer finishes a question, the client sends the question text to `POST /session/:id/supportive/hints` and receives up to three concise hints for structuring the next answer. Second, live feedback: after the client detects completion of a candidate answer (from transcript role transitions), it sends the previous question and answer to `POST /session/:id/supportive/feedback` and receives one short coaching message formatted for toast presentation. These supportive calls use `gpt-4o-mini` with structured JSON responses, and token usage for these calls is recorded via the backend usage endpoint. The implementation also includes guards to skip hints for introductory questions and to avoid generating feedback for extremely short or trivial answers.

VIII. EVIDENCE-BASED FEEDBACK PIPELINE

The implemented system runs three complementary analysis tracks over interview data: transcript analysis, audio analysis, and vision analysis. These tracks are integrated through a common persistence and reporting layer (filesystem-backed session archives under `server/reports`) rather than being isolated experiments.

A. Transcript Analysis

Transcript feedback is produced in two complementary stages. First, at session finalization the Node.js backend generates an initial report via a transcript evaluator component. Second, a dedicated Python transcript analyzer produces the structured transcript artifact (`transcript_report.json`) that powers per-question scoring, categorization, and higher-level recommendation grouping in the UI and history views.

The Python pipeline starts by normalizing the raw transcript into stable question-answer (QA) units and applying lightweight cleanup to reduce fragmentation and remove non-substantive turns. In particular, the pipeline detects and excludes setup/meta interactions (e.g., greetings, “hazırsanız başlayalım”, or closing pleasantries) so that scoring focuses on interview substance rather than dialogue management.

For each interviewer question, the analyzer assigns a `questionType` through a dedicated resolution layer. The resolver first applies rule-based meta detection; if a question is identified as setup/closing, it is labeled `meta` and excluded from scoring. Otherwise, semantic retrieval is performed using Qdrant over a small, seeded set of labeled example questions. The resolver retrieves the nearest examples (filtered by interview type), aggregates similarity scores per label to estimate confidence, and selects the best label when confidence is sufficient. If retrieval confidence is low, the system falls back to an LLM-based classifier to resolve ambiguous cases. The supported labels include `self_presentation`, `motivation`, `behavioral`, `experience`, `technical_knowledge`, `technical_experience`, and `problem_solving` (with `meta` excluded from scoring).

After question typing, the Python analyzer computes per-QA rule-based metrics in addition to any LLM-produced narrative interpretation. These metrics include relevance and clarity signals (e.g., lexical overlap and surface-level structure features), depth and example/evidence strength, and technical accuracy when applicable. Each QA entry is also annotated with visibility and scoring flags (e.g., `visibleInReport`, `excludedFromOverall`) to ensure that the overall score reflects interview substance rather than meta-dialogue.

Finally, recommendation generation is explicitly structured. The transcript artifact groups suggestions into three user-facing horizons: *Bir Sonraki Mülakatta* (immediately actionable tactics), *Performans Geliştirme* (mid-term interview technique improvement), and *Çalışma Planı* (topics to study/practice).

B. Audio Analysis

Audio analysis is implemented in the FastAPI-served Python service and is executed incrementally during the interview as candidate answer audios are uploaded. Each uploaded answer is converted to WAV and analyzed as an individual segment with `finalize_report=false`, producing evolving per-segment outputs in `audio_segments.json`. At session finalization, the system generates the final user-facing audio report (`audio_report.json`) without re-processing the full set of raw audio files.

During startup, the audio module initializes two main models: a Turkish clarity/intelligibility model (`mpoyraz/wav2vec2-xls-r-300m-cv7-turkish`) and an emotion classifier from the SUPERB benchmark (`superb/wav2vec2-base-superb-er`). Each answer is loaded at 16 kHz and analyzed across three signal families: (1) emotion distribution via `AutoModelForAudioClassification`, (2) clarity via CTC confidence using a Wav2Vec2 model, and (3) speaking behavior features estimated with Librosa (e.g., non-mute intervals via `librosa.effects.split` with

`top_db=25` and onset/peak statistics used to approximate speech rate).

The final audio report combines deterministic scores with an LLM-interpreted narrative layer. It includes an overall audio score, dominant/secondary emotion tendencies, tone distribution summaries, a `speechSummary` field, and practical recommendations (e.g., next-interview tactics and medium-term development). The pipeline also supports provisional (non-final) audio artifacts so the feedback page can correctly represent “pending” behavior until `audio_report.json` is completed.

C. Vision Analysis

Vision analysis is split into (i) a browser-side sampler and overlay stabilizer and (ii) a server-side analyzer that accumulates session-level visual signals. On the client, the vision sampling layer captures a JPEG frame approximately every 900 ms, encodes it as Base64, and uploads it to the backend via `POST /session/:id/vision/frame`. To stabilize the user-facing overlay, bounding boxes are smoothed using a blending factor of 0.55, and the last valid box may be retained for up to 650 ms when detection temporarily drops.

On the server, each ingested frame is analyzed by a Python vision module. The implementation prefers MediaPipe face detection and falls back to OpenCV Haar cascades when MediaPipe is unavailable or does not yield a face. The analyzer returns detector diagnostics and signals such as face count, bounding box coordinates, and eye-visibility proxies. The backend maintains a runtime vision sub-state that accumulates sampled frames, detected-face ratios, average face area and center offset, head-movement estimates, warning/danger counters, and whether supportive overlay guidance was shown.

At report finalization, these stored visual signals are summarized into scores such as face presence, focus/attention stability, centering, steadiness, and tension-related indicators (e.g., visual tension, attention risk, movement risk, and eye tension). A finalized vision report artifact (`vision_report.json`) is produced for the UI, and the system degrades gracefully when MediaPipe is unavailable because diagnostics and fallback reasons are explicitly recorded.

D. Aggregation, Reporting, and History

The backend composes analysis outputs into a single report view (served via `GET /session/:id/report`) rather than exposing independent analysis endpoints. The report response is assembled by loading persisted feedback artifacts from the filesystem archive and merging them with any in-memory report state when available. Concretely, the response includes transcript text (`transcript.txt`), transcript analysis (`transcript_report.json`), raw incremental audio outputs (`audio_segments.json`), the finalized audio report (`audio_report.json`), raw vision

runtime payload (`vision_frames.json`), and the finalized vision report (`vision_report.json`). It also includes score metadata (a score schema for UI rendering) and analysis-status flags derived from which artifacts exist. Operational metadata such as token usage and estimated cost is included when present in the in-memory session report; for purely archived reconstructions these fields may be `null`.

A filesystem-backed archive persists session artifacts under a session-specific directory `server/reports/S-*/`. These files are consumed by the History page to list sessions (via `GET /reports`) and compute cross-session insights (via `GET /reports/history-insights`). The history insight logic loads recent reports, groups transcript QA evaluations by `questionType`, and derives trend metrics only for categories that are present across the selected recent reports, avoiding trends computed from sparse labels. Table IV summarizes the main persisted artifacts used by reporting and history features.

IX. TESTING, VERIFICATION, AND REQUIREMENT COVERAGE

A. Automated Test Baseline

The project currently provides a successful verification baseline combining automated smoke testing and requirement-oriented assessment. The automated smoke suite passed 43 tests across 17 test files, including 18 client-side tests in 8 files and 25 server-side tests in 9 files [5]. Moreover, all 23 requirement-oriented test cases presented in this section were evaluated as passed, based on the appropriate combination of automated tests, manual checks, and targeted validation scenarios. Table V summarizes the executed smoke-test baseline across the client and server layers.

B. Automated Test Inventory

The client-side smoke suite covers the following observable behaviors:

- The application-level smoke tests verify that the interface can progress through the major states of setup, preview, interview, feedback, and history access.
- The setup-page and setup-form smoke tests verify that session preparation is triggered correctly and that the session cannot be started before mandatory consent is granted.
- The preview-page smoke tests verify preview-question loading, configuration update behavior, forward navigation into the interview stage, and backward navigation to setup.
- The interview-page smoke tests verify supportive hint and feedback requests, immediate report generation after interview completion, and incremental candidate-audio uploads with duplicate suppression.
- The client API smoke tests verify backend routing for session creation, preview-question generation, session

configuration updates, report polling, and history-related requests.

- The feedback-page smoke tests verify asynchronous readiness behavior, including visible status progression while analysis artifacts arrive and the continued pending state of audio analysis until the final audio report is available.
- The history-page smoke tests verify that prior reports can be listed and that a selected report can be opened from the history view.

The server-side smoke suite covers complementary backend behavior:

- The route smoke tests verify endpoint exposure, life-cycle mapping, and validation behavior for invalid session configurations.
- The guardrail smoke tests verify mandatory consent enforcement and valid-state checks before a session may start.
- The backend-orchestrator smoke tests verify first-question initialization, transcript fallback from candidate audio, duplicate-aware incremental audio ingestion, hydration of partial legacy runtime state, successful report generation under partial signals, and propagation of token-usage and cost metadata into the final report.
- The analysis-client smoke tests verify graceful degradation when transcript or vision analysis services are unavailable or time out.
- The prompt-template smoke tests verify supportive-mode guidance, constructive feedback language, and hidden time-management instructions in both technical and HR interview contexts.
- The realtime-session payload smoke tests verify that the realtime update builder includes turn detection and interviewer voice output in the generated session payload.
- The AI-service gateway smoke tests verify redirect-style supportive feedback for off-topic candidate answers.
- The transcript-evaluator smoke tests verify time-bound overrun detection when answers exceed the allocated response window.
- The realtime gateway smoke tests verify both successful SDP exchange and explicit failure signaling when the realtime endpoint is unreachable.

Taken together, these tests go beyond superficial rendering checks. They exercise asynchronous readiness logic, duplicate suppression, fallback paths, route validation, supportive guidance behavior, and graceful degradation under dependency failure.

C. Requirement Satisfaction Assessment

Based on the current verification baseline, Table VI summarizes the requirement areas for which supporting evidence was obtained through automated testing and manual verification.

Table IV
PERSISTED SESSION ARTIFACTS (FILESYSTEM ARCHIVE)

Artifact	Project-level purpose
session_config.json	Persist role, mode, interview type, and configuration for reproducibility and history metadata.
transcript.txt	Store the flattened human-readable transcript.
transcript_report.json	Store structured transcript analysis (QA scoring, labels, recommendations, and metadata).
audio_segments.json	Store incremental per-answer audio analysis outputs and segment metadata.
audio_report.json	Store the finalized user-facing audio interpretation (with completed gating).
vision_frames.json	Store accumulated vision samples and runtime vision metrics/diagnostics.
vision_report.json	Store the finalized vision report used by the feedback UI.
cv/ (optional)	Persist uploaded CV PDF and derived artifacts (e.g., extracted text and candidate_brief.json) when provided.

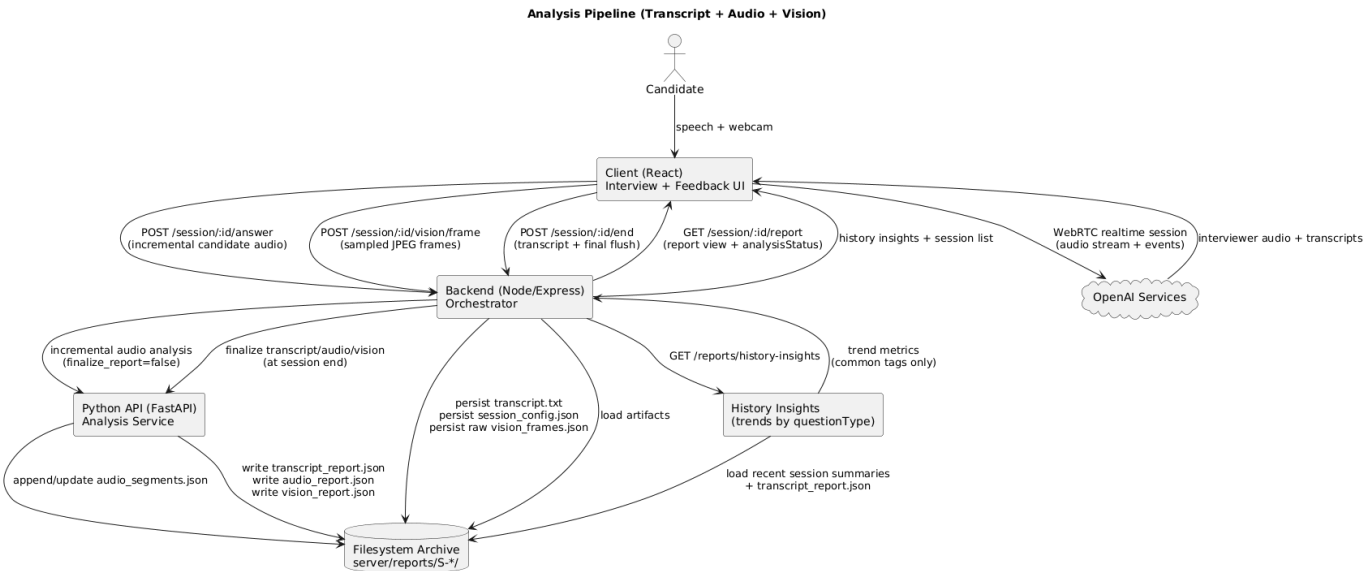


Figure 2. Analysis Pipeline

Table V
EXECUTED AUTOMATED SMOKE BASELINE

Layer	Test Files	Passing Tests	Test IDs	Primary Coverage Focus
Client	8	18	UTC-06, UTC-08, ITC-03, STC-01, STC-02, STC-03, STC-04, STC-05, UAT-01, UAT-02	UI flow, consent gating, setup, preview, interview actions, history, and report polling
Server	9	25	UTC-01, UTC-02, UTC-03, UTC-04, UTC-05, UTC-07, ITC-01, ITC-02, ITC-04, PTC-01, PTC-02, PTC-03, PTC-04	Routes, validation, orchestration, prompt construction, realtime payloads, degradation, and guardrails
Total	17	43	UTC-01-08, ITC-01-04, STC-01-05, PTC-01-04, UAT-01-02	Smoke-level functional confidence across the main application workflow

Table VI
REQUIREMENT-ORIENTED TEST RESULTS

Test ID	Description	Input(s)	Expected Output	Result
UTC-01	Validate interview configuration	Valid/invalid config DTOs	Valid config accepted; invalid config rejected with validation errors	Passed
UTC-02	Time-bound prompting on long answers	Transcript duration exceeds limit	Guardrail applies time-bound prompting guidance	Passed
UTC-03	Off-topic detection in supportive mode	Off-topic question/transcript pair	Session guidance redirects candidate back to topic	Passed
UTC-04	Uncertainty detection in supportive mode	“I don’t know” / “I’m not sure” style answers	Supportive hint or follow-up is returned without revealing the answer	Passed
UTC-05	Graceful degradation with missing behavioral signals	Session turns with audio only	Report still generated with partial behavioral section	Passed
UTC-06	No API key in client; backend-only AI access	Frontend scan and gateway mock	No client-side API key; AI access routed through backend	Passed
UTC-07	STT pipeline processes recorded responses	Recorded .webm audio	Transcript generated and stored for the response	Passed
UTC-08	Realtime audio response via WebRTC	Active AI interviewer response	Audio streamed through WebRTC without error	Passed
ITC-01	Session creation API stores config	POST /api/session	200 OK, sessionId returned, config stored	Passed
ITC-02	Session start returns context-aware first question	POST /api/session /{id}/start	First question matches context/difficulty; audio output produced	Passed
ITC-03	Audio pipeline triggers next system action	Frontend WebRTC audio stream	Transcript processed, guardrails evaluated, next response generated	Passed
ITC-04	Supportive behaviors work end-to-end	Supportive session with off-topic/uncertain input	Redirect or supportive follow-up triggered correctly	Passed
STC-01	Neutral session completes and generates report	Full neutral-mode UI flow	Session completes, report shown, duration within target	Passed
STC-02	Supportive session shows redirect and assistance	Supportive flow with scripted responses	Guidance triggered while flow remains coherent	Passed
STC-03	Microphone permission required	Mic denied / allowed	Denied blocks start; allowed enables start	Passed
STC-04	UI navigation across main screens	Full user flow	Setup, Interview, and Feedback screens reachable	Passed
STC-05	Audio-based behavioral signals appear in report	Session with speech signals	Report includes pace, pause, and filler metrics	Passed
PTC-01	Turn latency meets real-time expectation	Scripted turn timing	p95 latency remains within threshold	Passed
PTC-02	Post-session feedback generation time is acceptable	End session and measure completion time	Report generated within target time bound	Passed
PTC-03	Graceful degradation under analyzer failure	Forced behavioral-analysis failure	Session completes and report still generated	Passed
PTC-04	Redundant external AI calls are limited	Session call-count measurement	AI calls remain within defined cap	Passed
UAT-01	Guided flow is easy to start	Real-user setup flow	Interview starts quickly with low confusion	Passed
UAT-02	Feedback is constructive and safe	User review of generated report	Tone remains supportive and actionable	Passed

In addition to the automated smoke baseline, several important requirement areas were also evaluated through manual testing and confirmed during system-level verification:

- Real microphone capture, real camera capture, and actual browser permission behavior were manually tested and confirmed in the running application.
- Live WebRTC media playback with an actual interviewer voice was manually verified through end-to-end execution in the browser environment.
- End-to-end OpenAI-backed execution under live network conditions was manually tested and confirmed using the deployed service flow.
- Performance-related requirements, including response latency, turn timing, and report turnaround time, were manually observed and validated during execution-based testing.
- Runtime behavior beyond the automated smoke suite, including operational session handling and report generation flow, was manually checked and approved under realistic usage conditions.

Accordingly, the appropriate interpretation is that the project is supported by both automated smoke evidence and manual verification results. The automated suite provides strong evidence for core interaction flow, orchestration logic, readiness handling, and resilience paths, while manual testing confirms the practical operation of hardware-dependent, browser-dependent, realtime, and live-service behaviors.

D. Test Assessment Summary

The verification results indicate a successful overall test outcome. All 23 requirement-oriented test cases and all 43 automated smoke tests were completed successfully, and no failed test cases were observed during the evaluated baseline.

Although no critical failures were encountered, manual verification and execution-based observation revealed several practical areas for improvement. The most notable issues were related to asynchronous report readiness behavior and temporary delays caused by external analysis-service dependencies. These issues did not prevent successful session completion, but they indicate opportunities to improve system robustness and user-perceived responsiveness.

Based on the current test outcomes, the most relevant directions for future improvement are:

- replacing polling-based report updates with event-driven or push-based completion signaling,
- expanding browser-level end-to-end automated testing for live media workflows,
- strengthening persistence and recovery mechanisms for longer-running sessions, and
- reducing operational dependence on external services through additional caching, fallback, or recovery strategies.

E. Current Engineering Risks and Testing Limitations

Although the current verification baseline is successful, several engineering limitations remain relevant. First, the automated suite is primarily smoke-oriented rather than a full browser-media end-to-end validation suite. Second, external AI and analysis services are abstracted in automated tests, even though they remain operational dependencies in realistic deployments. Third, report readiness is driven by asynchronous artifact completion, and the current user-facing flow still relies on polling rather than push-based completion signaling. Fourth, while the project includes a Dockerized multi-service runtime for more reproducible deployment, broader production-scale concerns such as orchestration hardening, secret management, and long-running operational monitoring remain outside the present implementation scope. Finally, while core runtime behavior was validated through both automated and manual testing, persistence and recovery characteristics under long-running or failure-prone production conditions have not yet been evaluated as extensively as the main functional workflow.

X. RESULTS AND DISCUSSION

A. Delivered Technical Capabilities

Based on the implemented project and the verified system behavior, the project currently delivers a substantial set of technical capabilities. The system supports configurable HR and technical interview scenarios, together with both supportive and neutral interaction modes. It provides Turkish-language interviewer behavior, incorporates CV-based personalization into interview preparation, and enables live interview interaction through a WebRTC-based communication flow. In addition, the platform collects candidate transcripts and answer audio, performs transcript-, audio-, and vision-based analysis, stores generated session outputs for later access, and supports history-oriented report retrieval together with aggregated insight generation [1]–[4].

B. Architectural Strengths

One of the strongest properties of the system is its explicit coordination of multiple heterogeneous components within a single interview lifecycle. Browser-side media capture, server-side orchestration, AI-driven prompting, analysis services, and report persistence are separated into clear functional modules while still contributing to one coherent end-user workflow. This modular structure improves maintainability and makes the implementation easier to reason about.

The architecture also reflects several practical engineering strengths. Session guardrails are enforced before interview start, candidate audio can be uploaded incrementally instead of only at the end of the session, duplicate answer-audio submissions are filtered, and report generation can progress even when some analysis artifacts arrive later than others. In addition, fallback and degradation paths

are implemented so that the system can continue operating in a controlled manner when some external analysis services are unavailable.

C. Observed Limitations

Despite the breadth of implemented functionality, the current system still exhibits several practical engineering limitations. During active operation, parts of the runtime continue to rely on in-memory state, which constrains persistence and recovery robustness under more demanding deployment conditions. In addition, some analysis artifacts are produced asynchronously, so the feedback workflow currently depends on polling rather than push-based completion signaling. The client-side history mechanism is also primarily application-state-driven instead of URL-oriented, which reduces flexibility for deep-linking and direct navigation.

From an evaluation perspective, the project is supported by both automated smoke testing and manual verification. However, this evidence should be interpreted as strong functional validation rather than exhaustive production-level assurance. In particular, full browser-media end-to-end behavior, live external-service interaction, long-running persistence guarantees, and recovery under sustained real-world load would still benefit from broader deployment-scale testing. These limitations do not indicate conceptual weaknesses in the architecture; rather, they reflect the current engineering boundaries of the implemented system.

XI. CONTEMPORARY ISSUES RELATED TO AI-BASED INTERVIEW SYSTEMS

AI-based interviewing systems are currently discussed not only as technical tools but also as socio-technical systems with significant ethical and practical implications. One major issue is fairness. When AI models are used to interpret speech, language, or visual signals, there is a risk that different users may be treated unequally due to bias in training data, modeling assumptions, or evaluation criteria [11]–[14]. In the context of interview-related systems, such concerns become especially important because communication style, visual presentation, and linguistic variation can differ substantially across users.

A second contemporary issue is trust and transparency. Users may benefit from AI-generated interview guidance, but they may also be uncertain about how feedback is produced or how much confidence should be placed in automated interpretations. Recent literature on AI-enabled interviews emphasizes that perceived fairness, trust, and explainability strongly affect user acceptance of such systems [8], [9]. For this reason, AI-assisted interview platforms should be designed so that generated feedback is interpreted as decision support or practice assistance, rather than unquestionable judgment.

A third issue concerns privacy and responsible data use. Systems that process transcript, audio, and visual data

necessarily handle sensitive personal information. This creates obligations related to informed consent, secure storage, limited retention, and clear communication about how user data is processed. In addition, the broader literature on algorithmic hiring warns that even supportive AI systems can shape user behavior and perceptions in ways that require ethical reflection [7], [10].

The implemented project does not function as an automated hiring decision system; instead, it is intended as a practice and feedback platform. Nevertheless, these contemporary issues remain relevant because they influence how such a system should be engineered, evaluated, and communicated to users. Accordingly, the project should be understood within the broader context of fairness-aware, privacy-aware, and user-centered AI system design.

XII. IMPACT ANALYSIS

The implemented system has implications across global, economic, environmental, and societal dimensions. By combining artificial intelligence, web technologies, and multimodal analysis, it provides a scalable and accessible approach to interview preparation. At the same time, its deployment raises practical concerns related to fairness, privacy, and responsible use [7], [8], [10]. This section discusses both the positive contributions and the broader implications of the project.

A. Global Impact

The platform reflects a broader global shift toward AI-assisted professional preparation and digitally mediated career development. Because the system is web-based and does not depend on in-person interview coaching, it can potentially be used by candidates in different geographic locations with fewer logistical constraints. In this sense, the project supports wider access to structured interview practice and aligns with the global trend toward remote, technology-supported learning and assessment environments [18], [19].

B. Economic Impact

The system can improve economic efficiency by reducing dependence on costly interview-preparation resources such as private coaching services or paid mock-interview platforms. Because the platform automates routine practice sessions and is accessible online, users can prepare more flexibly and at lower cost. From an operational perspective, the use of automated interviewing and feedback generation also reduces reliance on human interviewers for repetitive training scenarios, which supports scalability. More broadly, the system may contribute to improved job readiness and reduce the time required for effective interview preparation.

C. Environmental Impact

As a web-based platform, the system supports remote preparation and reduces reliance on physical materials and face-to-face practice environments. This can decrease

paper consumption, transportation requirements, and the environmental costs associated with travel. Although the platform depends on digital infrastructure and computational resources, its overall footprint may still remain lower than that of traditional preparation approaches that require physical attendance or printed resources. For this reason, the system can be regarded as a comparatively resource-efficient alternative for many interview-practice scenarios.

D. Societal Impact

The system has the potential to create positive societal value by making structured interview preparation more accessible to a wider range of users. Candidates from different educational, social, and economic backgrounds may benefit from guided practice without depending on professional networks or expensive services. In addition, the platform supports the development of communication, self-expression, and interview-performance skills through repeated practice and personalized feedback.

At the same time, the societal use of such a platform requires careful attention to ethical concerns. Because the system processes audio, transcript, and visual data, explicit consent, secure handling of user data, and clear privacy safeguards are necessary. Similarly, the use of AI-based evaluation introduces risks related to bias, fairness, and over-reliance on automated judgment [11]–[14]. These concerns do not eliminate the value of the system, but they do require responsible design, transparent usage, and careful interpretation of generated feedback.

XIII. CONCLUSION

This project delivers a well-designed AI-assisted mock interview platform rather than a simple conversational interface. The implemented system integrates interview configuration, prompt-driven interviewer behavior, supportive and neutral interaction modes, WebRTC-based live interviewing, CV-aware personalization, multimodal evidence collection, report generation, session archival, and history-oriented analytics within a single end-to-end workflow.

From a technical perspective, the strongest aspect of the platform is its orchestration architecture. The client, backend, and analysis subsystems are separated clearly enough to remain maintainable, while still being integrated closely enough to provide a unified interview and feedback experience. The repository also demonstrates a practical containerized deployment approach through Docker-based service separation for the client, server, and Python analysis layer. The verification results presented in this report further indicate that the core user flow, API behavior, guardrail logic, fallback handling, and analysis-readiness behavior are functioning as intended in the current project state.

Accordingly, the project can be concluded to represent a full-stack mock interview platform that combines mode-

aware interviewer behavior, practical system orchestration, and evidence-oriented feedback generation. At the same time, as discussed in the evaluation sections, broader production-scale validation would further strengthen confidence in long-running persistence, realtime robustness, and live external-service behavior.

REFERENCES

- [1] Client source code, “Application flow, page structure, and interface components,” project repository, 2026.
- [2] Backend source code, “Server orchestration, routing, validation, and report persistence,” project repository, 2026.
- [3] Realtime interview and prompting modules, “Interviewer behavior, session configuration, and supportive-mode logic,” project repository, 2026.
- [4] Analysis modules, “Transcript, audio, and vision analysis implementation,” project repository, 2026.
- [5] Automated verification assets, “Client and server smoke-test suites,” project repository, 2026.
- [6] Deployment and containerization assets, “Docker Compose configuration, service Dockerfiles, and Docker setup documentation,” project repository, 2026.
- [7] I. Fister and G. K. Thiruvathukal, “Artificial Intelligence Employment Interviews: Examining Limitations, Biases, and Perceptions,” *Computer*, vol. 57, no. 6, pp. 48–57, 2024, doi: 10.1109/MC.2024.3404669.
- [8] X. Luo *et al.*, “Understanding fairness and trust in AI-enabled interviews,” *Humanities and Social Sciences Communications*, vol. 12, no. 1, pp. 1–12, 2025, doi: 10.1057/s41599-025-05607-z.
- [9] H. Y. Suen, M. Y. C. Chen, and S. H. Lu, “Applicant reactions to AI-based video interviews: A review and research agenda,” *Technological Forecasting and Social Change*, vol. 197, p. 123684, 2024, doi: 10.1016/j.techfore.2023.123684.
- [10] A. Fabris *et al.*, “Algorithmic hiring systems: A multidisciplinary perspective,” *ACM Computing Surveys*, 2025, doi: 10.1145/3696457.
- [11] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.
- [12] R. Binns, “Fairness in machine learning: Lessons from political philosophy,” *Proceedings of Machine Learning Research*, vol. 81, pp. 149–159, 2018.
- [13] J. Buolamwini and T. Gebru, “Gender shades: Intersectional accuracy disparities in commercial gender classification,” *Proceedings of Machine Learning Research*, vol. 81, pp. 1–15, 2018.
- [14] M. Buyl and T. De Bie, “On the inherent limitations of algorithmic fairness,” *arXiv preprint arXiv:2212.06495*, 2022.
- [15] Y. Zhang *et al.*, “AI-assisted interviewing: Exploring LLM-based question generation,” *arXiv preprint arXiv:2509.12709*, 2025.
- [16] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 2021.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [18] HireVue Inc., “HireVue AI Interview Platform Overview,” 2023. [Online]. Available: <https://www.hirevue.com>
- [19] Pymetrics Inc., “Pymetrics AI Hiring Platform,” 2023. [Online]. Available: <https://www.pymetrics.com>
- [20] OpenAI, “GPT-4 Technical Report,” 2024. [Online]. Available: <https://openai.com/research>

APPENDIX A

USER INTERFACE SCREENSHOTS

This appendix presents representative user interface screens from the implemented system. The figures illustrate both the main interview workflow and the post-session analysis interfaces.

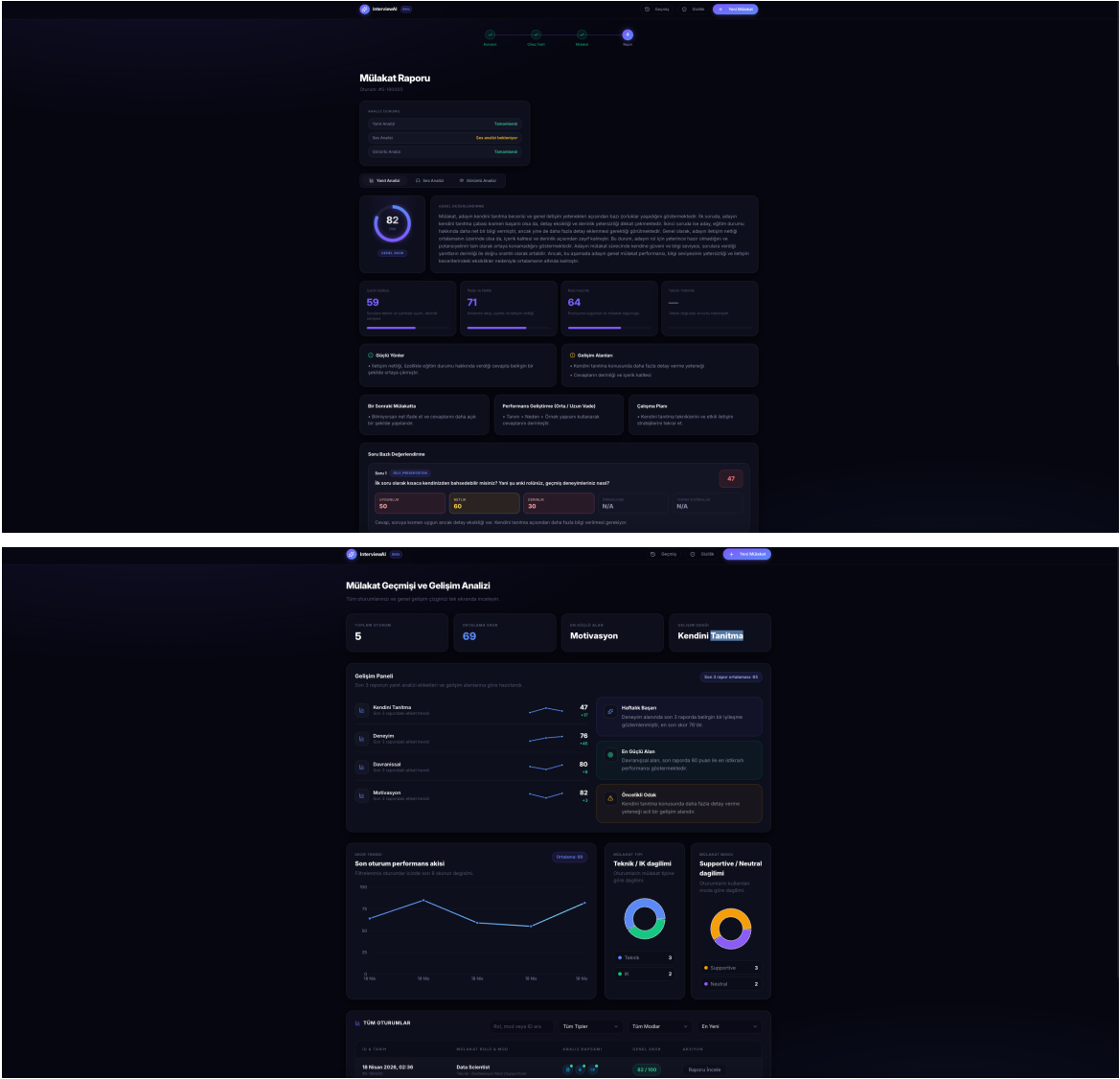


Figure 4. Post-session interfaces. From top to bottom: (1) feedback/report screen showing multimodal analysis results, and (2) history screen displaying previous sessions and performance trends.